

ENHANCED EMERGENCY RESPONSE SYSTEM

19I503-INTERNET OF THINGS

PRESENTED BY
22I248-NAVEEN SS
23I431-CHARAN K
23I433-SACHIN KARTHIK V
23I434-SIVARAMAN S
23I435-VIJAYABASKAR R

CONTENTS

01

INTRODUCTION

02

COMPONENTS
REQUIRED

03

ARCHITECTURE OF
ESP8266

04

CONNECTIONS

05

WORKING
PRINCIPLES

06

ARDUINO SKETCH &
WEB APPLICATION

07

APPLICATIONS

08

CONCLUSION

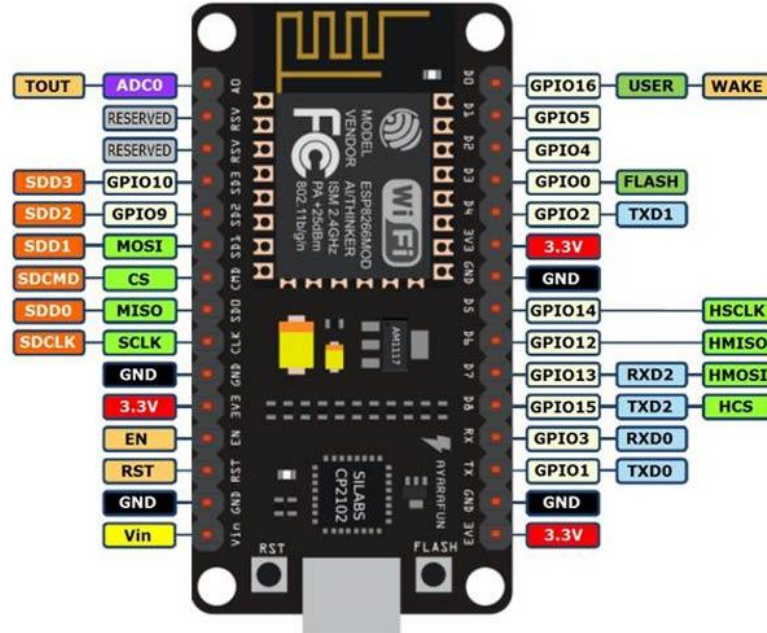
COMPONENTS REQUIRED

1.	ESP8266 (NodeMCU)
2.	MQ2 GAS SENSOR
3.	DHT11 SENSOR (Temperature&Humidity)
4.	IR SENSOR
5.	BUZZER
6.	CONNECTING WIRES & BREADBOARD

INTRODUCTION

- IoT-based Emergency System:** Monitors temperature, humidity, gas, and smoke levels to detect emergencies.
- Multi-sensor Setup:** Uses DHT11 for temperature/humidity, MQ-2 for gas/smoke, and an IR sensor for motion detection.
- Real-time Alerts:** Triggers alarms with LED and buzzer when dangerous conditions are detected.
- Cloud Integration:** Sends sensor data to ThingSpeak for remote monitoring and analysis.
- Web application:** integration allows real-time monitoring of sensor data, providing users with instant access to environmental conditions.

ARCHITECTURE OF ESP8266

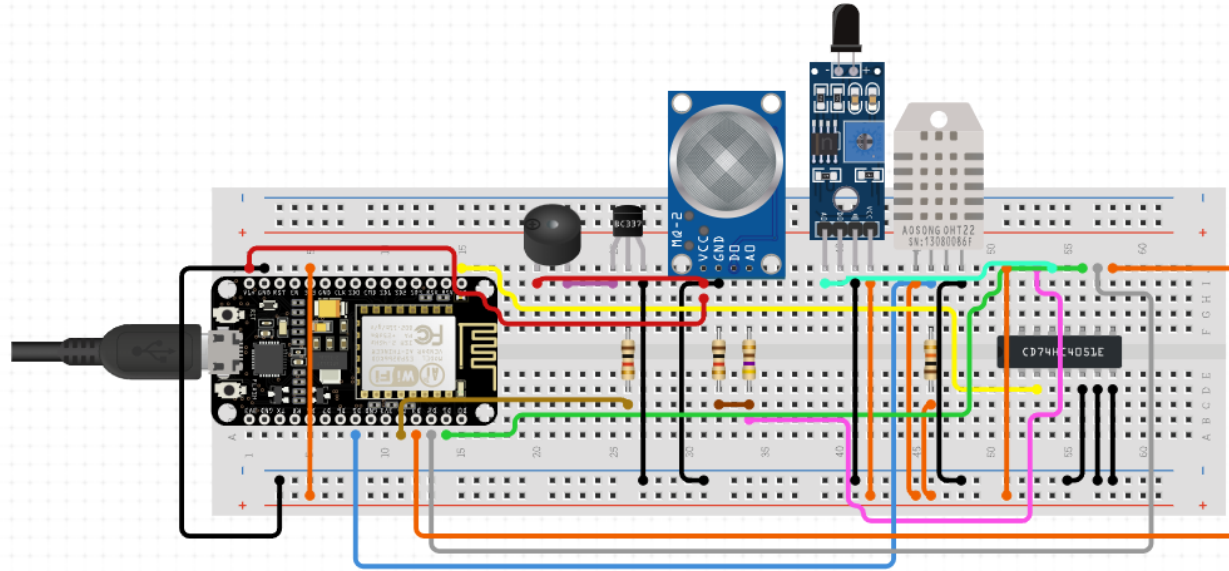


ARCHITECTURE OF ESP8266

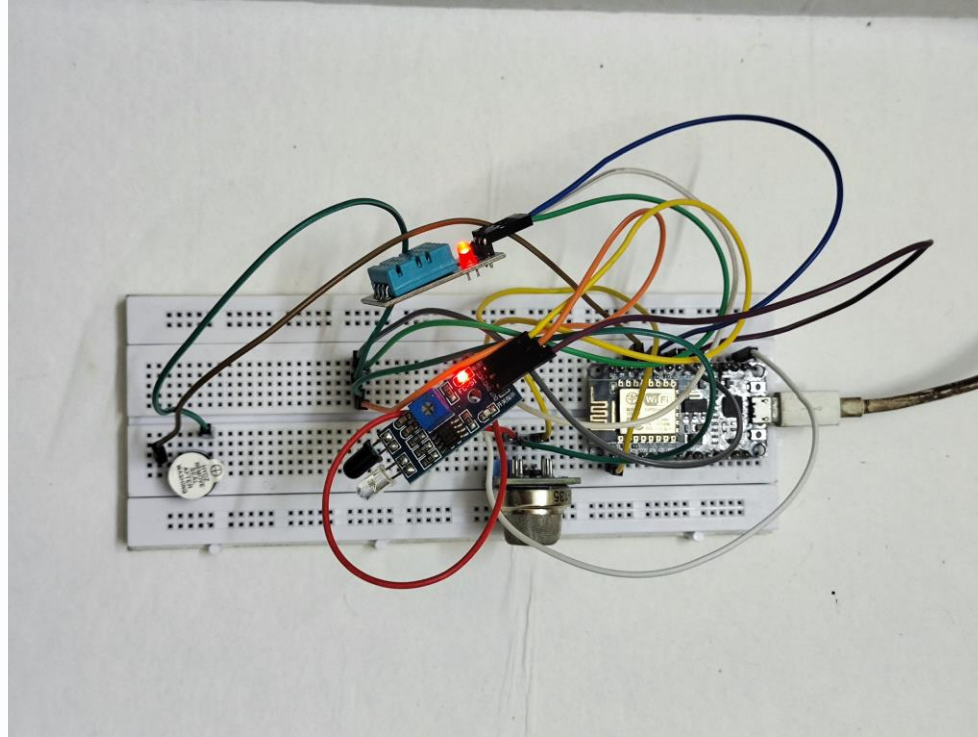
The ESP8266 microcontroller is designed for efficient wireless communication and processing:

- **CPU:** A low-power 32-bit processor (80-160 MHz) that handles Wi-Fi, GPIO control, and user applications.
- **Wi-Fi Module:** Supports 2.4 GHz Wi-Fi for network connectivity, with an integrated TCP/IP stack for web communication.
- **Memory:** 160 KB RAM for processing, and up to 4 MB flash storage for firmware and code.
- **GPIO:** 16 pins for connecting sensors and devices, supporting I2C, SPI, UART, and PWM.
- **Power Management:** Low-power modes for battery use, operates at 3.3V.
- **ADC:** One analog input pin for sensors.

CONNECTIONS



CONNECTIONS



WORKING PRINCIPLES

- Sensor Monitoring:** The system continuously monitors environmental data using sensors like DHT11 for temperature and humidity, MQ-2 for smoke/gas, and an IR sensor for object detection.
- Threshold Detection:** The system compares real-time sensor values against predefined threshold values (e.g., temperature above 30°C, high gas levels). If any values exceed these limits, an alert is triggered.
- Buzzer and LED Activation:** Upon detecting hazardous conditions (such as smoke or high temperature), the buzzer and LED are activated to provide immediate local warnings.
- Data Upload to ThingSpeak:** Sensor data is transmitted to the ThingSpeak IoT platform, where it can be stored, analyzed, and visualized in real-time.
- Web App Integration:** A web application is developed to link with ThingSpeak, allowing users to view live readings and trends from the sensors, enhancing remote monitoring capabilities.
- Real-Time Remote Monitoring:** The uploaded data and web app interface enable users to remotely monitor environmental conditions and respond quickly during emergencies.

ARDUINO SKETCH

```
#include <ESP8266WiFi.h>
#include <WiFiClient.h>
#include <ESP8266HTTPClient.h>
#include <DHT.h>

// Define pins for sensors and outputs
#define DHTPIN D4           // Pin where the DHT11 is connected
#define DHTTYPE DHT11      // DHT11 sensor
#define BUZZER_PIN D2      // Pin for the buzzer
#define LED_PIN D5         // Pin for the LED
#define SMOKE_SENSOR_PIN A0 // Analog pin for the smoke sensor (MQ-2)
#define GAS_SENSOR_PIN A0  // Analog pin for the gas sensor (MQ-135) - same as smoke sensor for testing
#define IRSENSORPIN D6     // IR sensor pin

// Initialize the DHT11 sensor
DHT dht(DHTPIN, DHTTYPE);

// ThingSpeak API credentials
const char* writeAPIKey = "DPPM6BOSWIR21G86"; // Replace with your ThingSpeak API key
const char* server = "api.thingspeak.com";

// WiFi credentials
const char* ssid = "12341234"; // Your WiFi SSID
const char* password = "123412345"; // Your WiFi Password

// Set threshold values
const float TEMP_THRESHOLD = 30.0; // Temperature threshold in °C
const float HUMIDITY_THRESHOLD = 70.0; // Humidity threshold in %
const int MQ2_THRESHOLD = 300; // MQ-2 gas/smoke threshold (adjust based on calibration)

WiFiClient client;
```

ARDUINO SKETCH

```
void setup() {  
  // Initialize serial communication  
  Serial.begin(115200);  
  
  // Initialize sensor and output pins  
  pinMode(BUZZER_PIN, OUTPUT);  
  pinMode(LED_PIN, OUTPUT);  
  pinMode(IRSENSORPIN, INPUT); // Set IR sensor pin as input  
  
  // Start DHT sensor  
  dht.begin();  
  
  // Connect to WiFi  
  WiFi.begin(ssid, password);  
  while (WiFi.status() != WL_CONNECTED) {  
    delay(1000);  
    Serial.println("Connecting to WiFi...");  
  }  
  Serial.println("Connected to WiFi");  
}  
  
void loop() {  
  // Read temperature and humidity from DHT11 sensor  
  float temperature = dht.readTemperature();  
  float humidity = dht.readHumidity();  
  
  // Read values from smoke sensor (MQ-2)  
  int smokeLevel = analogRead(SMOKE_SENSOR_PIN);  
  
  // Read IR sensor for object detection  
  int irValue = digitalRead(IRSENSORPIN);  
  int objectDetected = (irValue == LOW) ? 1 : 0; // 1 for detected, 0 for not detected
```

ARDUINO SKETCH

```
// Print sensor data to the serial monitor
Serial.print("Temperature: ");
Serial.print(temperature);
Serial.println(" °C");

Serial.print("Humidity: ");
Serial.print(humidity);
Serial.println(" %");

Serial.print("Smoke Level: ");
Serial.println(smokeLevel);

Serial.print("Object Detection: ");
Serial.println(objectDetected ? "Object detected" : "No object detected");

// Check thresholds and take action
if (temperature > TEMP_THRESHOLD || humidity > HUMIDITY_THRESHOLD || smokeLevel > MQ2_THRESHOLD) {
    Serial.println("WARNING: Threshold exceeded!");
    digitalWrite(BUZZER_PIN, HIGH);
    digitalWrite(LED_PIN, HIGH);

    // Send data to ThingSpeak
    sendToThingSpeak(temperature, humidity, smokeLevel, objectDetected);
} else {
    digitalWrite(BUZZER_PIN, LOW);
    digitalWrite(LED_PIN, LOW);
}

// Wait for 2 seconds before the next reading
delay(2000);
}

void sendToThingSpeak(float temperature, float humidity, int smokeLevel, int objectDetected) {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
```

ARDUINO SKETCH

```
// Build URL for the GET request
String url = "http://" + String(server) + "/update?api_key=" + String(writeAPIKey) +
"&field1=" + String(temperature) +
"&field2=" + String(humidity) +
"&field3=" + String(smokeLevel) +
"&field4=" + String(objectDetected); // New field for object detection

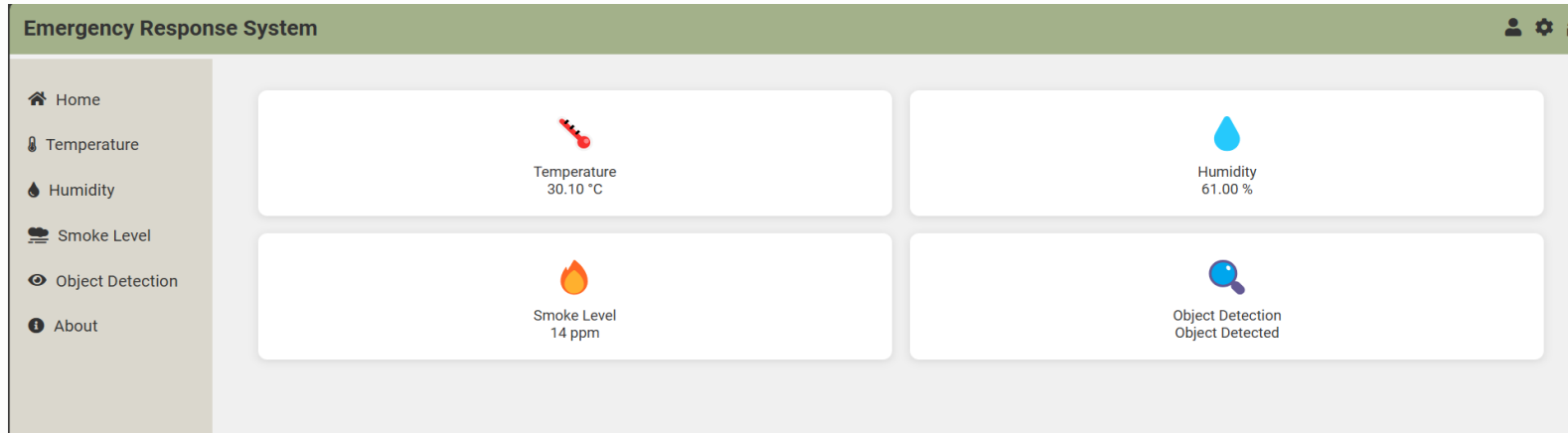
// Print URL to serial for debugging
Serial.print("Sending data to URL: ");
Serial.println(url);

// Send the request to ThingSpeak
http.begin(client, url);
http.setTimeout(10000); // Set timeout to 10 seconds
int httpResponseCode = http.GET();

// Check for HTTP errors
if (httpResponseCode > 0) {
  Serial.print("ThingSpeak Response code: ");
  Serial.println(httpResponseCode);
  if (httpResponseCode == 200) {
    Serial.println("Data sent successfully to ThingSpeak!");
  } else {
    Serial.println("Error: Non-200 response from ThingSpeak.");
  }
} else {
  Serial.print("Error sending data to ThingSpeak: ");
  Serial.println(http.errorToString(httpResponseCode));
}
http.end(); // Free resources
} else {
  Serial.println("WiFi not connected.");
}
}
```

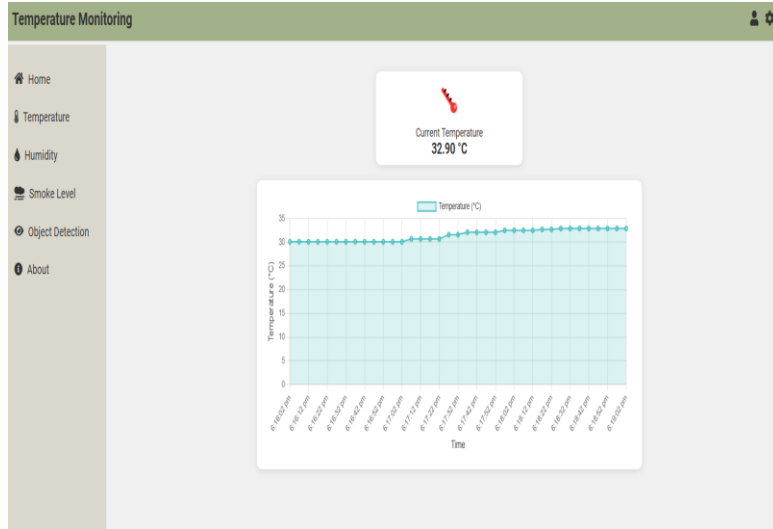
WEB APPLICATION

DASHBOARD

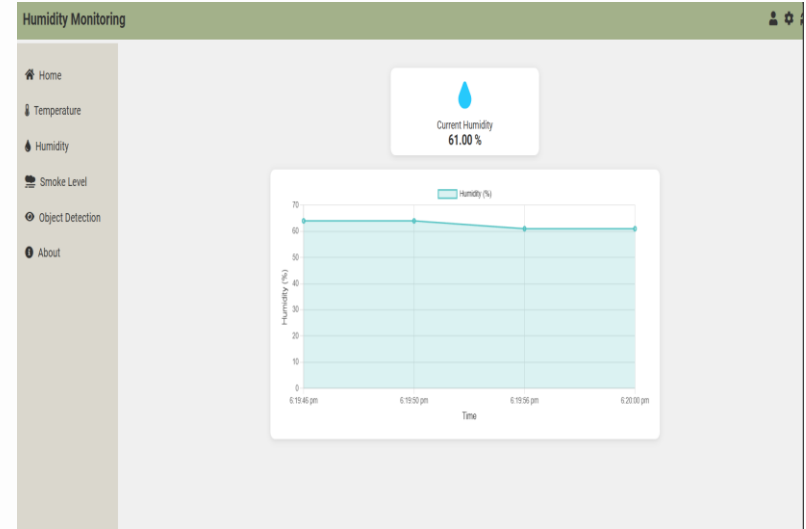


WEB APPLICATION

TEMPERATURE

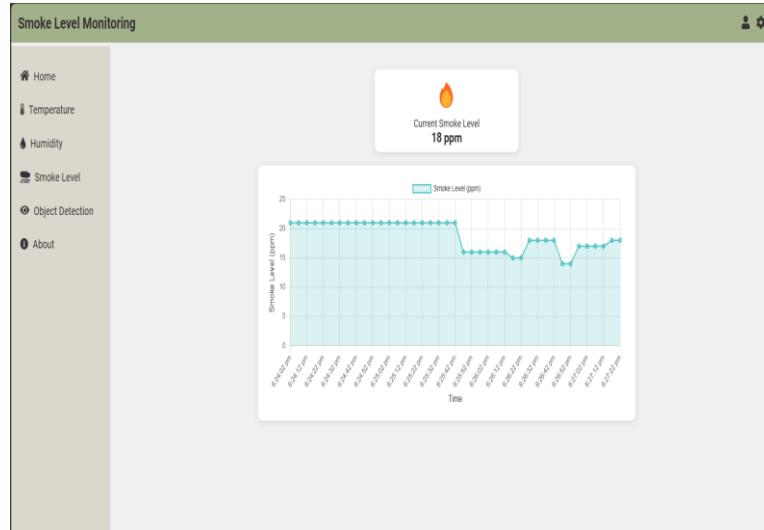


HUMIDITY



WEB APPLICATION

SMOKE LEVEL



COMPONENTS USED

About Our Project

Home Temperature Humidity Smoke Level Object Detection About

About the Sensors



DHT11 Sensor

The DHT11 is a reliable digital sensor used for measuring temperature and humidity. It combines a thermistor and a capacitive humidity sensor, providing accurate readings that are essential for climate control systems.

Applications include weather stations, HVAC systems, and indoor environmental monitoring. Its low power consumption makes it ideal for battery-operated devices.



MQ-2 Sensor

The MQ-2 sensor is a versatile gas sensor that detects various gases such as LPG, methane, and smoke. Utilizing a metal oxide semiconductor, it can sense gas concentrations in the air.

This sensor is widely used in gas leakage detection, fire alarm systems, and air quality monitoring, providing critical safety alerts in domestic and industrial settings.



Flame Sensor

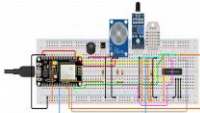
The flame sensor detects the presence of flames by measuring infrared radiation emitted by fire. It is highly sensitive to flame wavelengths, ensuring timely detection.

Common applications include fire safety systems, automated extinguishing systems, and industrial safety protocols, making it vital for emergency response scenarios.

WEB APPLICATION

ABOUT

Implementation Overview





Connecting the Sensor: We connected a DHT11 temperature and humidity sensor to our Arduino. We ensured the data pin was correctly wired to a digital pin on the Arduino for accurate readings.

Writing Arduino Code: Using the Arduino IDE, we wrote code that enables the Arduino to read data from the DHT11 sensor. We incorporated an ESP8266 module to send the data to ThingSpeak for cloud storage and analysis.

Setting Up ThingSpeak: We created an account on ThingSpeak and set up a channel to store our sensor data. We obtained the Channel ID and Write API Key, which are essential for sending our data to the cloud.

Creating a Webpage: We developed a simple HTML page integrated with JavaScript. This page fetches the sensor data from ThingSpeak and displays it in real time on our dashboard, allowing easy monitoring.

Deployment and Testing: After uploading the Arduino code, we opened our webpage to monitor the real-time sensor data. We tested the system thoroughly, adjusting thresholds and email notifications to ensure everything functions as intended.

About Our Project

- Home
- Temperature
- Humidity
- Smoke Level
- Object Detection
- About

Project Overview

In this project, we developed a comprehensive sensor data dashboard that utilizes a DHT11 temperature and humidity sensor, along with additional sensors for gas detection and flame monitoring, all integrated with a NodeMCU (ESP8266) microcontroller. By leveraging the capabilities of the ESP8266, we facilitated real-time data transmission to ThingSpeak, a cloud-based platform for IoT applications. Our custom HTML dashboard fetches and displays sensor readings, providing users with an intuitive interface to monitor environmental conditions. Furthermore, we implemented email notifications to alert users when sensor thresholds are exceeded, enhancing safety and responsiveness.

APPLICATIONS

- Smart Home Security:** Monitors smoke and gas levels for enhanced safety and immediate alerts to homeowners and emergency services.
- Industrial Safety Monitoring:** Detects hazardous conditions in factories to ensure worker safety and regulatory compliance.
- Environmental Monitoring:** Tracks soil moisture and air quality for better agricultural management and weather risk reduction.
- Disaster Management:** Provides real-time data during natural disasters to help emergency responders allocate resources effectively.
- Healthcare Facilities:** Ensures optimal temperature and humidity in hospitals to prevent pathogen growth, particularly in sensitive areas.
- Public Buildings and Infrastructure:** Detects emergencies like fires or gas leaks promptly to safeguard large groups in schools, malls, and transport systems.

CONCLUSION

The Enhanced Emergency Response System effectively monitors environmental conditions using DHT11, MQ-2, and IR sensors to detect hazards such as high temperatures and gas leaks. By employing threshold detection, the system triggers local alerts via buzzer and LED activation when predefined limits are exceeded.

Data is transmitted to the ThingSpeak IoT platform for real-time monitoring and visualization, and also provides useful insights about the situation with the help of the web application allowing users to respond quickly to emergencies. This project showcases the practical application of IoT technology in enhancing safety and emergency preparedness.

THANK YOU